



CompSci 230

Object Oriented Software Development

Java Basics



Review Quizzes

- ▶ Java statements end with _____.
 - ▶ the keyword end
 - ▶ a semicolon
 - ▶ a period
 - ▶ a right brace
- ▶ Which command executes the Java class file Welcome.class?
 - ▶ javac Welcome.class
 - ▶ java Welcome.class
 - ▶ java Welcome
 - ▶ run Welcome.class



Today's Agenda

► Topics:

- Variables and Literals
- Primitive Data Types
- Reference Data Types
- Using printf
- Type Conversions
- Prompting the User for Input

► Reading:

- Java how to program Late objects version (D & D)
 - Chapter 2
- The Java Tutorials
 - Lesson: Language Basics:Variables
 - <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>
- Welcome to Java for Python Programmers
 - <http://interactivepython.org/courselib/static/java4python/index.html>



1. Variables and Literals

Variables & Literals

- ▶ A **variable** is a named storage location in the computer's memory.
- ▶ A **literal** is a value that is written into the code of a program.
- ▶ Programmers determine the number and type of variables a program will need.

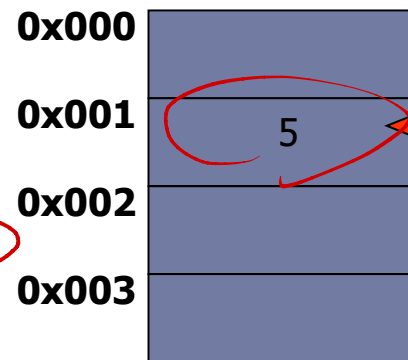
This line is called
a *variable declaration*.

```
int value;
```

The following line is known
as an assignment statement.

```
value = 5;
```

```
System.out.print("The value is ");  
System.out.println(value);
```



The value 5
is stored in
memory.

This is a string *literal*. It will be printed *as is*.

*Print content of
variable*

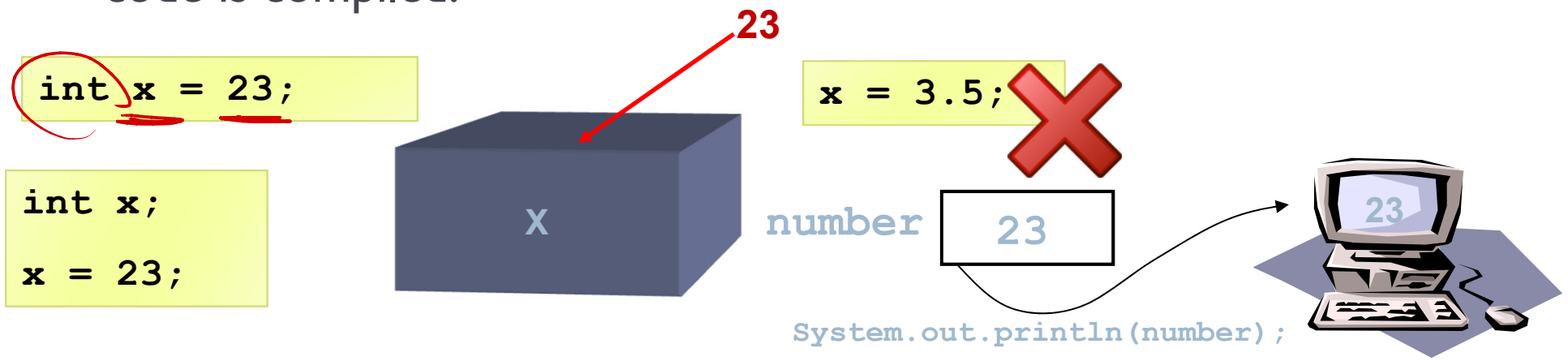
The integer 5 will
be printed out here.
Notice no quote marks?



1. Variables and Literals

Declare and initialise a variable

- ▶ Java is a statically typed language.
 - ▶ Variables must be declared before they can be used.
 - ▶ Once declared, they can then receive a value (initialization); however the value must be compatible with the variable's declared type.
 - ▶ Use an assignment statement to initialize the value
 - Left operand: must be a variable name; right operand: a literal or expression
 - ▶ Trying to use uninitialized variables will generate a Syntax Error when the code is compiled.





1. Variables and Literals

Identifiers

- ▶ Identifiers are programmer-defined names for:
 - ▶ classes
 - ▶ variables
 - ▶ methods
- ▶ Identifiers may not be any of the Java reserved keywords.
- ▶ Identifiers must follow certain rules:
 - ▶ An identifier may only contain:
 - ▶ letters a–z or A–Z,
 - ▶ the digits 0–9,
 - ▶ underscores (`_`), or
 - ▶ the dollar sign (`$`)
 - ▶ The first character may not be a digit.
 - ▶ Identifiers are case sensitive.
 - ▶ `itemsOrdered` is not the same as `itemsordered`.
 - ▶ Identifiers cannot include spaces.

```
double percentagePassing101, percentagePassing105;
```

```
boolean isTheLectureOverYet;
```



1. Variables and Literals

Variable Names

- ▶ Variable names should be descriptive.
- ▶ Descriptive names allow the code to be more readable; therefore, the code is more maintainable.

```
double salesTaxRate = 0.0725;
```

- ▶ Java programs should be self-documenting.
- ▶ Variable names should begin with a lower case letter and then switch to title case there after:

```
int caTaxRate;
```

- ▶ Class names should be all title case.

```
public class BigLittle
```

- ▶ More Java naming conventions can be found at:

<http://java.sun.com/docs/codeconv/html/CodeConventions.doc8.html>



1. Variables and Literals

Java Reserved keywords

abstract	double	instanceof	static
assert	else	int	strictfp
boolean	enum	interface	super
break	extends	long	switch
byte	false	native	synchronized
case	for	new	this
catch	final	null	throw
char	finally	package	throws
class	float	private	transient
const	goto	protected	true
continue	if	public	try
default	implements	return	void
do	import	short	volatile
			while



1. Variables and Literals

Creating Constants – final

- ▶ Constants are identifiers that can hold only a single value. But the value it stores cannot change once it is initialised.

```
final int DAYS_IN_YEAR = 365;  
final double GST_RATE = 0.125;
```

Rules:

- ▶ Constants are declared using the keyword final.
- ▶ Constants need not be initialized when declared; however, they must be initialized before they are used or a compiler error will be generated.
- ▶ Once initialized with a value, constants cannot be changed programmatically.

```
DAYS_IN_YEAR = 30;
```



```
System.out.println(DAYS_IN_YEAR);
```

365

- ▶ Use the following style guidelines for naming symbolic constants:
 - ▶ use all upper case letters
 - ▶ separate multiple words with an underscore



Exercise 1-2

- ▶ Which of the following is a valid identifier?

```
$234  
maxValue  
int  
-Week  
2radius
```

- ▶ Create an enum Weekday with constants for
 - ▶ Mon, Tue, Wed, Thur and Fri

```
Weekday a_day = Weekday.FRIDAY;  
System.out.println(a_day);
```



2. Primitive Data Types

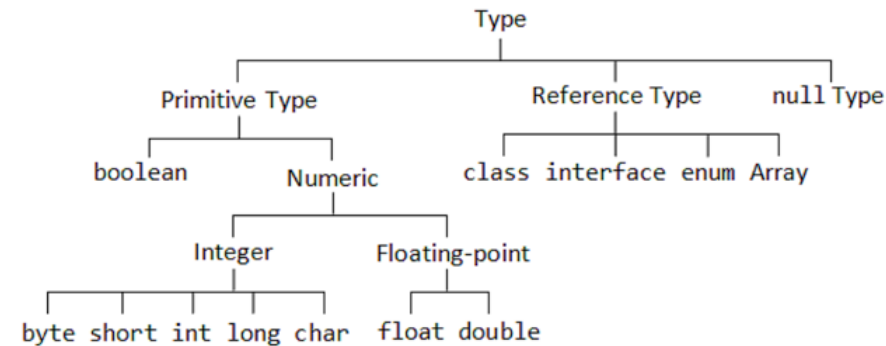
- ▶ Java is a strongly typed language.
 - ▶ Java requires all variables to have a type.
- ▶ Java's types are divided into primitive types and reference types.
- ▶ Most basic data types like integers and floating point numbers are not objects. They are called primitive data types. Operations on the primitives are fast
 - ▶ There are eight primitive types :

`byte, short, int, long, float, double, char, boolean`

- ▶ Integers (whole numbers):
- ▶ Floating-point numbers (decimal numbers):
- ▶ Characters (symbol in character set (text)):
- ▶ boolean (true or false values):

```
int intA = 15;  
int intB = 10;
```

intA	15
intB	10



Single character
T or F



2.Primitive Data Types

Numeric Data Types

byte	1 byte	Integers in the range -128 to +127
short	2 bytes	Integers in the range of -32,768 to +32,767
int	4 bytes	Integers in the range of -2,147,483,648 to +2,147,483,647
long	8 bytes	Integers in the range of -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807
float	4 bytes	Floating-point numbers in the range of $\pm 3.410^{-38}$ to ± 3.41038 , with 7 digits of accuracy
double	8 bytes	Floating-point numbers in the range of $\pm 1.710^{-308}$ to ± 1.710308 , with 15 digits of accuracy



2.Primitive Data Types

Numeric Data Types

▶ Integer Data Types:

- ▶ byte, short, int, and long are all integer data types.
- ▶ Integer data types cannot hold numbers that have a decimal point in them.
- ▶ Integers embedded into Java source code are called *integer literals*

12

Do not use commas, decimal points, or leading zeros

▶ Floating Point Data Types

- ▶ Data types that allow fractional values are called floating-point numbers.
- ▶ In Java there are two data types that can represent floating-point numbers.
 - ▶ float - also called single precision (7 decimal points).
 - ▶ double - also called double precision (15 decimal points).
- ▶ floating point numbers are embedded into Java source code they are called floating point literals.
- ▶ Note: The default type for floating point literals is double.

12.7

0.0



2.Primitive Data Types

Floating Point Literals

- ▶ Note: The default type for floating point literals is double.
 - ▶ i.e. A double value is not compatible with a float variable because of its size and precision.

```
float number;  
number = 23.5; // Error!
```



```
float number;  
number = 23.5F; // This will work.
```

make a number a float by
appending the letter f or F

- ▶ Literals cannot contain embedded currency symbols or commas.

```
grossPay = $1,257.00; // ERROR!  
grossPay = 1257.00;   // Correct.
```

- ▶ Floating-point literals can be represented in scientific notation.
 - ▶ 47,281.97 is equal to 4.728197×10^4
 - ▶ Java uses E notation to represent values in scientific notation.
 - ▶ 4.728197×10^4 is expressed as 4.728197E4.

```
double distance, mass;  
distance = 1.495979E11;  
mass = 1.989E30;
```



2. Primitive Data Types

boolean & char

▶ The boolean Data Type

- ▶ The Java boolean data type can have two possible values.
 - ▶ true
 - ▶ false

▶ The char Data Type

- ▶ The Java char data type provides access to single characters.

- ▶ char literals are enclosed in single quote marks.

'a', 'Z', '\n'

- ▶ Don't confuse char literals with string literals.

- char literals are enclosed in single quotes.
 - String literals are enclosed in double quotes.

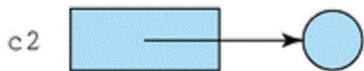
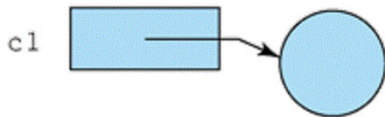
"hello"

Char

String

3. Reference Types

- ▶ **Reference Types:** *class, string, Array*
 - ▶ Any data type that is not a primitive type is a reference type
 - ▶ Programs use variables of reference types (normally called references) to store the addresses of objects in the computer's memory.
 - ▶ Such a variable is said to refer to an object in the program.
 - ▶ If a variable is not initialized explicitly, the variable is given the special value **null**.



```
Point b = new Point( 10, 20 );
```

16

<u>b: Point</u>
x = 10
y = 20

Reference type	Brief description
Array	Provides a fixed-size data structure that stores data elements of the same type.
Class	Designed to provide inheritance, polymorphism, and encapsulation. Usually models something in the real world and consists of a set of values that holds data and a set of methods that operates on the data.
Enumeration	A reference for a set of objects that represents a related set of choices.

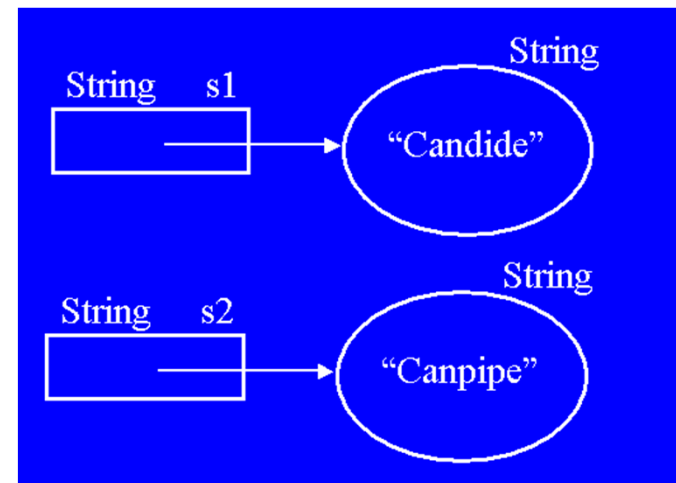


2. Primitive Types vs. Reference Types

- ▶ To call an object's methods, you need a reference to the object.
 - ▶ If an object's method requires additional data to perform its task, then you'd pass arguments in the method call.

```
String s1 = new String("candide");  
String s2 = s1.replace('d', 'p');
```

or
String s1 = "candide";



- ▶ Primitive-type variables do not refer to objects, so such variables cannot be used to invoke methods.



3.Reference Type Enum

- ▶ An enum is a special "class" that represents a group of **constants**
 - ▶ To create an enum, use the enum keyword (instead of class or interface), and separate the constants with a comma. Note that they should be in uppercase letters:

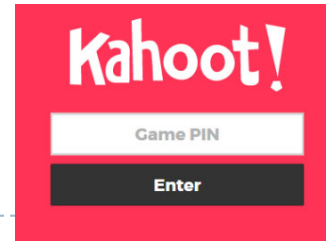
```
enum Level {  
    LOW,  
    MEDIUM,  
    HIGH  
}
```

```
Level myVar = Level.MEDIUM;  
Day myDay = Day.Sunday;
```

```
public enum Day {  
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY  
}
```



3. Reference Type The String Class



- ▶ Java has **no primitive data type** that holds a series of characters.
- ▶ The String class from the Java standard library is used for this purpose.
- ▶ In order to be useful, the a variable must be created to reference a String object.
- ▶ Notice the S in String is upper case.

```
String number;
```

```
String greeting = "Hello World!";  
char c = 'H';
```



Reference Types Vs Primitive Types

Reference types	Primitive types
Unlimited number of reference types, as they are defined by the user.	Consists of boolean and numeric types: char, byte, short, int, long, float, and double.
Memory location stores a reference to the data.	Memory location stores actual data held by the primitive type.
When a reference type is assigned to another reference type, both will point to the same object.	When a value of a primitive is assigned to another variable of the same type, a copy is made.
When an object is passed into a method, the called method can change the contents of the object passed to it but not the address of the object.	When a primitive is passed into a method, only a copy of the primitive is passed. The called method does not have access to the original primitive value and therefore cannot change it. The called method can change the copied value.



4. Console Output

Printf for Java Format String

- ▶ printf allows the programmer to specify textual representations of data using two parameters:

- ▶ a format string (template)
- ▶ an argument list

- ▶ Syntax:

```
System.out.printf(format_string [, one or more values]);
```

- ▶ To display values you embed formats into the printf string.
 - ▶ A format starts with “%” and conversion characters.
 - ▶ After the string, you list the variables, constants or values that you want mapped to the formats in the format string.

- ▶ General Syntax:

- ▶ `%[flags][width][.precision][argsize]typechar`

Conversion Character	Output
%c	character
%d	Decimal integer
%f	Floating point numbers
%s	String
%n	A new line character



4. Console Output

%s

- ▶ The width of the String and the justification can be specified:

- ▶ Flags:

- ▶ '-' Result is left-aligned in the field. *

- ▶ Width

- ▶ the minimum number of characters

- ▶ left-padded with spaces if value < minimum length

- ▶ will not be truncated if the converted value exceeds the minimum length

```
String words = new String("hello");  
System.out.println("123456789012345");
```

```
System.out.printf("%s.....%n", words);  
System.out.printf("%10s.....%n", words);  
System.out.printf("%-10s.....%n", words);
```

"%10s" - up to 10 **right-justified** chars, pad fewer than 10 chars on the left with blank spaces

"%-10s" - up to 10 **left-justified** chars, pad fewer than 10 chars on the right with blank spaces

```
123456789012345  
hello.....  
        hello.....  
hello      .....
```



4. Console Output

%d

```
int num1 = 345;  
int num2 = -6;
```

- ▶ It is the specifier for an int value.

- ▶ Flag

- ▶ '-' Result is left-aligned in the field.

- ▶ Width

- ▶ the minimum number of digits

- ▶ left-padded with spaces if value < minimum length

- ▶ will not be truncated if the converted value exceeds the minimum length

```
123456789012345678901234567890  
345...-6...  
          345...      -6...  
345          ...-6      ...
```

```
System.out.println("123456789012345678901234567890");  
System.out.printf("%d...%d...%n", num1, num2);  
System.out.printf("%10d...%7d...%n", num1, num2);  
System.out.printf("%-10d...%-7d...%n", num1, num2);
```

“%10d” – Display up to 10 **right-justified** digits, pad fewer than 10 chars on the left with blank spaces

“%-10d” - Display up to 10 **left-justified** digits, pad fewer than 10 chars on the right with blank spaces



4. Console Output

`%f`

```
double numd1 = 345.48;  
double numd2 = -0.12345678;
```

- ▶ It is the specifier for a double value. The width, the number of decimal places and the justification of the output can be specified.

- ▶ Flag

- ▶ '-' Result is left-aligned in the field.
 - ▶ '+' Non-negative values begin with a plus character ('+').

- ▶ Width

- ▶ the minimum number of digits
 - ▶ left-padded with spaces if value < minimum length
 - ▶ will not be truncated if the converted value exceeds the minimum length

- ▶ Precision

- ▶ number of fractional digits after the decimal point. The converted value will be rounded if necessary.

```
System.out.printf("%f...%f...%n", numd1, numd2);  
System.out.printf("%15f...%11f...%n", numd1, numd2);  
System.out.printf("%.2f...%.3f...%n", numd1, numd2);  
System.out.printf("%-15.1f...%-11.2f...%n", numd1, numd2);
```

```
123456789012345678901234567890  
345.480000...-0.123457...  
      345.480000... -0.123457...  
345.48...-0.123...  
345.5          ...-0.12      ...
```

“%-15.1f” Display up to 15 left-justified characters, pad fewer than 15 characters on the right with blank spaces (i.e., field width is 15) and display exactly 1 digit after the decimal point (.1)



Exercise 3-4

- ▶ What is the output of the following code fragment?

```
System.out.println("1234567890123456789012345");  
double x = 27.5, y = 33.785;  
System.out.printf("x=%10.4f y=%10.2f%n", x, y);
```

- ▶ Complete the following program which prints the area and perimeter of a circle. Print the result to 2 decimal places.

```
public class MyProgram {  
    public static void main(String[] args) {  
        double radius = 7.5;
```



5.Type Conversion

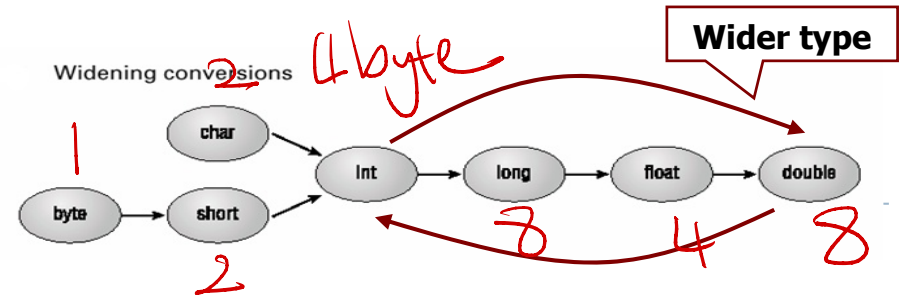
- ▶ Java supports two types of castings – *primitive* data type casting and *reference* type casting.
 - ▶ Reference type casting is nothing but assigning one Java object to another object. It comes with very strict rules and is explained clearly in Object Casting.
- ▶ Java data type casting comes with 3 flavours.
 - ▶ Implicit casting
 - ▶ Explicit casting
 - ▶ Boolean casting.



5.Type Conversion Implicit Casting

► Implicit casting (widening conversion)

- A data type of lower size (occupying less memory) is assigned to a data type of higher size.
- This is done implicitly by the JVM. The lower size is widened to higher size. This is also named as automatic type conversion.



```
double d = 4.9;  
int i = 10;  
double d1, d2;
```

— 8 byte
— 4 byte

Smaller → bigger only

► Wider assignment

```
d1 = i;
```

Assignment conversion
d1 = 10.0

► Wider Casting

- Casting a value to a wider value is always permitted but never required. However, explicitly casting can make your code more readable

```
d2 = (double) i;
```

Casting conversion (optional)
d2 = 10.0

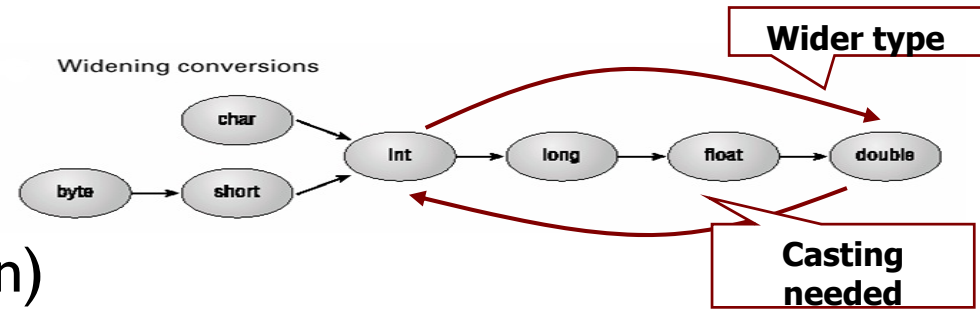
5.Type Conversion Explicit Casting

► Explicit casting (narrowing conversion)

- A data type of higher size (occupying more memory) cannot be assigned to a data type of lower size.
 - This is not done implicitly by the JVM and requires **explicit casting**;
 - A casting operation to be performed by the programmer.
 - The higher size is narrowed to lower size.

```
double d = 4.9;  
int i = 10;  
int i1, i2;
```

byte



Illegal conversion of a quart to a cup, with loss of data



► Narrow assignment – Compile-time error

```
//i1 = d;
```

Assignment conversion
Compile-time error

► Narrow Casting – Loss of information

- You must use an explicit cast to convert a value in a large type to a smaller type, or else converting that value might result in a **loss of precision**.

```
i2 = (int) d;
```

Narrow Casting: i2=4

NOTE: Everything beyond the decimal point will be truncated



5.Type Conversion Boolean Casting

► Boolean casting

- A boolean value cannot be assigned to any other data type.
 - Except boolean, all the remaining 7 data types can be assigned to one another either implicitly or explicitly; but boolean cannot.
 - i.e. boolean is incompatible for conversion.

```
boolean x = true;  
int y = x;
```

error: incompatible types: boolean cannot be converted to int

```
int y = x;  
      ^
```

| error



5.Type Conversion Example

► char <=> int

```
char ch1 = 'A';  
int y = ch1;  
System.out.println(y);
```

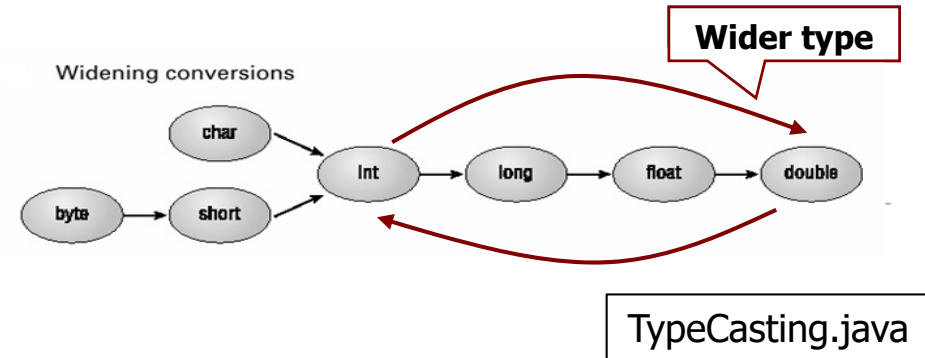
```
int d2 = 66;  
char ch2 = (char) d2;  
System.out.println(ch2);
```

65
B

► char <=> double

```
char ch3 = 'a';  
double z = ch3;  
System.out.println(z);
```

97.0



ASCII Character Codes Table & Cheat Sheet

Special Chars	Upper Case	Lower Case	Extended ASCII
9 \t (Tab)	65 A	97 a	128 C €
10 \n (NL)	66 B	98 b	129 D 
13 \r (CR)	67 C	99 c	130 , ‚
32 Space	68 D	100 d	131 f ƒ
33 !	69 E	101 e	132 " „
34 "	70 F	102 f	133 ... …
35 #	71 G	103 g	134 † †
36 \$	72 H	104 h	135 ‡ ‡
37 %	73 I	105 i	136 ^ ˆ
38 &	74 J	106 j	137 %oo ‰
39 '	75 K	107 k	138 Š Š
40 (	76 L	108 l	139 ‹ ‹
41)	77 M	109 m	140 Œ Œ
42 *	78 N	110 n	141 ♦ 
43 +	79 O	111 o	...



5.Type Conversion

Converting Strings to numbers

- ▶ String to int: use the static `Integer.parseInt()` method from the `Integer` class:

```
String input = "12";  
int num1 = Integer.parseInt(input);
```

num1

12

- ▶ String to double: use the static `Double.parseDouble()` method from the `Double` class:

```
String guess = "3.1415";  
double num2 = Double.parseDouble(guess);
```

num2

3.1415



Summary

	char	int	float	double
char char ch = 'A';	-	int x = ch; 65	float f = ch; 65.0	double d = ch; 65.0
int int x = 65;	char ch = (char)x; A	-	float f = x; 65.0	double d = x; 65.0
float float f = 65F;	char ch = (char) f; A	int x = (int) f; 65	-	double d = f; 65.0
double double d = 65;	char ch = (char) d; A	int x = (int) d; 65	float f = (float) d; 65.0	-



Exercise 5



▶ Complete the following code fragment:

▶ Convert from int to String

```
int value = 12;  
String myString = ...
```

▶ Convert from String to int

```
String myValue = "123";  
int myInt = ...
```



Exercise 6-7

► The value of the expression `(int) 27.6` evaluates to:

- a) 28
- b) 27
- c) 26
- d) Error
- e) 6

► What is the output of the following code fragment?

```
String phrase = "123";  
int c = Integer.parseInt(phrase);  
System.out.println(c*2);
```

6. Prompting the User for Input

- ▶ A Java program can obtain inputs from the console through the keyboard.
- ▶ The variable named **System.in** represents the keyboard
 - ▶ There is a lot of work that the computer must do to read in a number
- ▶ The Java programming language provides a collection of methods stored in the **Scanner** class that perform read operations.

java.util

Class Scanner

[java.lang.Object](#)

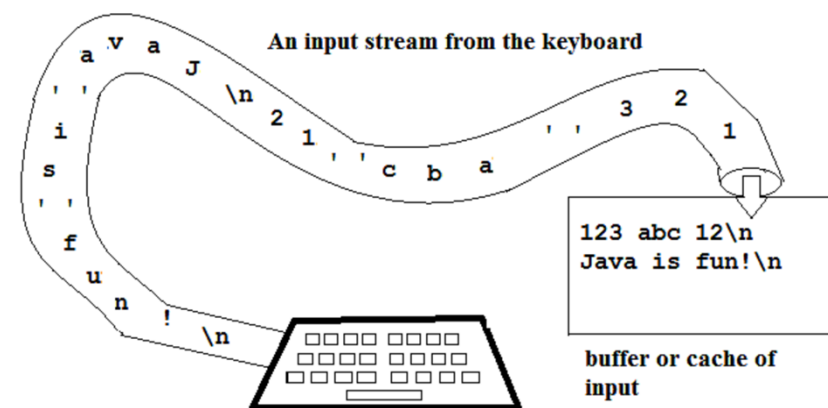
└ [java.util.Scanner](#)

All Implemented Interfaces:

[Iterator<String>](#)

```
public final class Scanner
extends Object
implements Iterator<String>
```

A simple text scanner which can parse primitive types and strings using regular expressions.





6. Prompting the User for Input Declaring and Creating a Scanner

► Importing the Scanner class definition

- Scanner is in a package named java.util

① `import java.util.Scanner;`

- To use Scanner, you must place the above line at the top of your program (before the public class header).

► Constructing a Scanner object to read console input:

`Scanner console = new Scanner(System.in);`

- The new keyword creates an object.
- Standard input object, System.in, enables applications to read bytes of data typed by the user.
- Scanner object translates these bytes into types that can be used in a program.



6.Prompting the User for Input

Scanner methods

- ▶ After having constructed the Scanner object named in:
 - ▶ Display a prompt: - A message telling the user what input to type.
 - ▶ Use one of the following methods to read from the keyboard:

Method	Description
<u>nextInt()</u>	reads a token of user input as an int
<u>nextDouble()</u>	reads a token of user input as a double
<u>next()</u>	reads a token of <u>user</u> input as a String
<u>nextLine()</u>	reads a line of user input as a String

- ▶ Each method waits until the user presses Enter.
- ▶ The value typed is returned.
- ▶ You must save (store) the number read in a variable with an assignment statement

```
System.out.print("How old are you? ");    // prompt
int age = console.nextInt() ;
System.out.println("You'll be 40 in " + (40 - age) + " years.");
```



6.Prompting the User for Input

Example 1

- ▶ Summarizes the programming steps to read in a number:
 - ▶ 1. import the Scanner class
 - ▶ 2. Construct a Scanner object
 - ▶ 3. Display a prompt message
 - ▶ 3. Define a variable to receive the value
 - ▶ 4. Read input

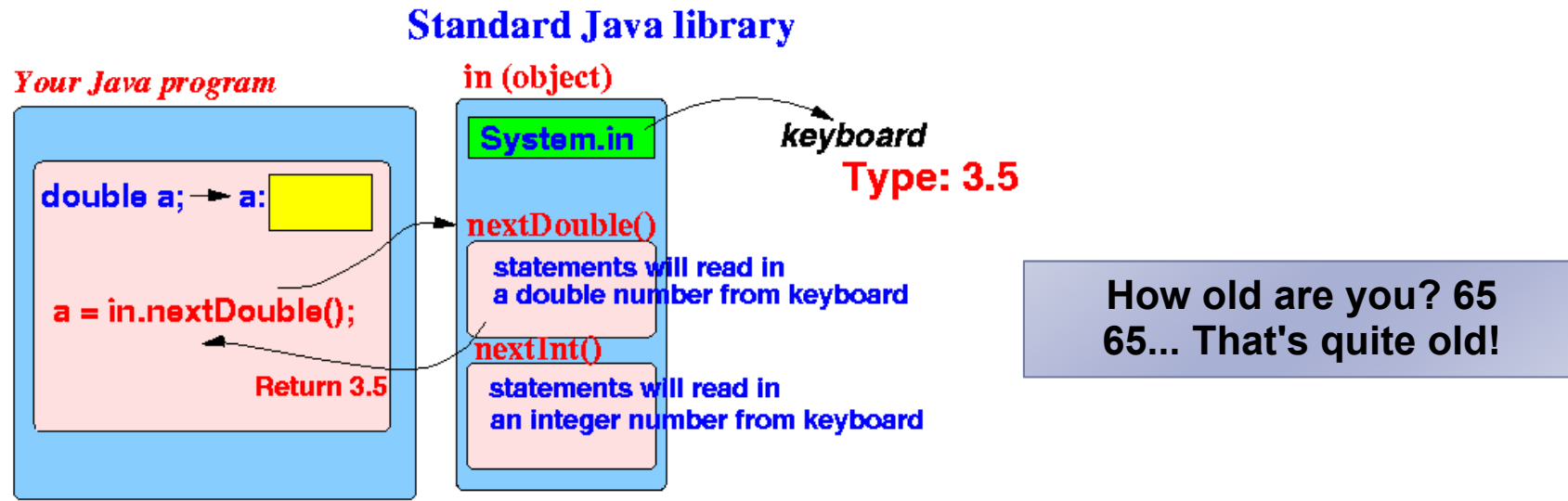
How old are you? 65
65... That's quite old!

```
import java.util.Scanner;
public class L02Code {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        System.out.print("How old are you? ");
        int age = console.nextInt();
        System.out.println(age + "... That's quite old!");
    }
}
```



6.Prompting the User for Input Example 2

- ▶ Reading a floating point number from the keyboard:



```
import java.util.Scanner;
...
Scanner console = new Scanner(System.in);
...
double x = console.nextDouble();
```



Exercise 8-9

- ▶ Write a program that prompts the user to enter an integer as a radius and calculates the area and perimeter of a circle. Print the result to two decimal places.

```
Enter a radius:
12
Perimeter is = 75.40
Area is = 452.39
```

- ▶ Write a Java program that prompts the user to enter two integers and calculates the sum of two input integers.

```
public class MyProgram {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int number1 = input.nextInt();
        int number2 = input.nextInt();
        ...
    }
}
```

Note: Create ONE scanner object only!